# Mathematical methods: Computational Tensor Algebra

## Reliable calculations with LLM assistance

Obinna Umeh

PhD Lectures 2026

2026






# What you should be able to do

- Translate a tensor statement into explicit software definitions.
- Run a small xAct calculation and explain each declaration.
- Test a result using identities and a simple independent example.
- Use an LLM to help write code while retaining a reproducible calculation.

| | |
|---|---|
| **0–13 min** | Motivation, tools and conventions |
| **13–30 min** | Mathematica, xAct and an exercise |
| **30–42 min** | Verification and research workflow |
| **42–45 min** | Discussion |

# Why learn a tensor algebra tool in 2026?

- An LLM can help draft calculations, explain code and suggest checks.
- A tensor system executes explicit operations on declared mathematical objects.
- Tool access can let an LLM run the same calculation. You still need to inspect the definitions and evidence.
- Research requires a result you can rerun, modify and defend.

**Learning target:** enough fluency to specify a problem, inspect generated code and verify its output.

Source: ToRA (ICLR 2024), an example of LLM reasoning with external tools. This is not a tensor-algebra benchmark.

# Benefits and costs of computer algebra

**Benefits**

- Exact symbolic manipulation under stated assumptions.
- Consistent handling of dummy indices and declared symmetries.
- Repeated calculations and expressions too large to manage by hand.
- Executable records that support checks and collaboration.

**Costs and limitations**

- Learning the representation, debugging and checking conventions.
- Memory, runtime, expression growth and software access.
- Incorrect assumptions produce incorrect physics. Software can have bugs.

# A useful division of work

| | |
|---|---|
| **Researcher** | Chooses geometry, conventions, approximation and physical interpretation. |
| **LLM** | Helps translate notation into code, explains errors and proposes tests. Check its suggested APIs. |
| **Tensor CAS** | Executes contractions, substitutions and simplifications using declared rules. |
| **Evidence** | Saved code, executed outputs, assumptions and independent checks support the result. |

Agreement between two chat responses supplies little independent evidence. A CAS calculation also needs a correct specification.

# Choosing a tool for the calculation

| **Tool** | **Relevant use** |
|---|---|
| Mathematica + xAct | Abstract tensors, differential geometry and GR perturbations. Mathematica requires suitable access. |
| Cadabra | Field theory, tensor identities, fermions and Clifford algebra. Open source. |
| Python + SymPy | Symbolic Python workflows, arrays and abstract tensor expressions. Open source. |

Today we use **Mathematica**, focusing on the **xAct framework**. Its four core packages support the extensions described next.

For cosmological perturbation theory, we use **xPand**.

Source: xAct documentation Cadabra documentation SymPy tensor documentation

# The four core packages of xAct

Four packages act as a kernel for the rest of the framework:

- **xCore**: generic programming tools.
- **xPerm**: manipulation of large groups of permutations.
- **xTensor**: abstract tensor computations, the flagship of the system.
- **xCoba**: component tensor computations.

Extensions build on this core. For cosmological perturbation theory, we use **xPand**, which builds on xTensor and xPert.

Source: xAct package catalogue and documentation

# Other packages include

- Invar: polynomial invariants of the Riemann tensor, by JMMG, David Yllanes, Renato Portugal and Leon Manssur.
- Spinors: spinor computations in GR, by Alfonso García-Parrado and JMMG.
- SymManipulator: Symmetrized expressions in xAct, by Thomas Bäckdahl.
- xTerior: Exterior calculus, by Alfonso García-Parrado and Leo C. Stein.
- SpinFrames: NP and GHP form of spinor equations, by Thomas Bäckdahl and Steffen Aksteiner.
- bimEX: $3+1$ Bimetric relativity, by Francesco Torsello.
- FieldsX: Fermions, gauge fields and BRST cohomology, by Markus B. Fröb.
- SymSpin: Symmetric spinors, by Steffen Aksteiner and Thomas Bäckdahl.
- HiGGS: Hamiltonian analysis of Poincaré gauge theory, by W. E. V. Barker.

# Other packages include (continued)

- xPert: high-order perturbation theory in GR, by David Brizuela, JMMG and Guillermo Mena Marugán.
- **xPand**: Cosmological perturbation theory, by Cyril Pitrou, Xavier Roy and Obinna Umeh.
- xIST/COPPER: General scalar-tensor theories and perturbations, by Johannes Noller.
- xPPN: Parametrized post-Newtonian formalism, by Manuel Hohmann.
- EFTofPNG: Effective field theory of post-Newtonian gravity, by Michele Levi and Jan Steinhoff.
- xTras: Additions to xAct, by Teake Nutma.

**We use xPand for cosmological perturbation theory.**

# Mathematica syntax needed today

```
f[x_] := x^2                    (* function definition *)
expr = (x + y)^2;               (* store, suppress output *)
Expand[expr]                    (* function arguments use [] *)
expr /. y -> 0                 (* apply a replacement rule *)
FullSimplify[Sqrt[x^2], Assumptions -> x > 0]
(* x *)
```

- = assigns a value, == forms an equation.
- := evaluates the right side when the definition is used.
- Start a fresh kernel before the demonstration. Earlier definitions can change later results.

Source: Wolfram Language: FullSimplify and assumptions

# The mathematical specification

Before asking any software to calculate:

- Manifold dimension, index types and tensor symmetries.
- Metric signature and connection, including torsion if present.
- Curvature convention and derivative order.
- Background, gauge, perturbative order and domain assumptions when relevant.

For today's example: a four-dimensional Lorentzian manifold, signature $(-,+,+,+)$, and the Levi–Civita connection.

$$\nabla_c g_{ab} = 0, \qquad S_{ab} = S_{ba}, \qquad F_{ab} = -F_{ba}.$$

Curvature convention: $[\nabla_c, \nabla_d] v^a = R^a{}_{bcd} v^b$. An abstract index labels a tensor slot. Components also require a basis or chart.

# A minimal xTensor session

```
Needs["xAct`xTensor`"];
DefManifold[M4, 4, {a, b, c, d, e, f}];
DefMetric[-1, g[-a, -b], CD, {";", "nab"}];
DefTensor[S[-a, -b], M4, Symmetric[{-a, -b}]];
DefTensor[F[-a, -b], M4, Antisymmetric[{-a, -b}]];
```

- `-a` denotes a covariant index. `a` denotes a contravariant index.
- `DefMetric` also sets up the metric connection and curvature objects.
- These declarations encode the mathematical assumptions. Review them before trusting a simplification.

Source: xTensor documentation: DefManifold, DefMetric and DefTensor

# A contraction that must vanish

$$S_{ab}F^{ab} = S_{ba}F^{ba} = -S_{ab}F^{ab} = 0.$$

The first equality renames dummy indices. The second uses the declared symmetries.

```
ToCanonical[S[-a, -b] F[a, b]]
(* 0 *)
```

```
ContractMetric[g[a, b] g[-a, -b]] // ToCanonical
(* 4 *)
```

**Question:** what changes if we never declare that $S_{ab}$ is symmetric?

# Exercise: removing the trace

For a symmetric tensor in four dimensions, define

$$Q_{ab} = S_{ab} - \frac{1}{4} g_{ab} g^{cd} S_{cd}.$$

1. Predict $g^{ab} Q_{ab}$ by hand.
2. Run the code and explain the output.
3. Replace 1/4 by 1/3. What residual should remain?

```
qexpr = S[-a, -b] - g[-a, -b] g[c, d] S[-c, -d]/4;
ContractMetric[g[a, b] qexpr] // ToCanonical
```

**Four minutes:** work in pairs. If you have no installation, audit the code and compute the residual by hand.

# Exercise solution and diagnosis

In dimension $n$, with coefficient $\alpha$,

$$g^{ab}Q_{ab} = (1 - \alpha n)S^{c}{}_{c}.$$

- $n = 4$, $\alpha = 1/4$: zero for every symmetric $S_{ab}$.
- $n = 4$, $\alpha = 1/3$: $-S^{c}{}_{c}/3$, generally nonzero.
- The coefficient $1/3$ is appropriate for a tensor projected into a three-dimensional spatial rest space, using $h_{ab}$.

The software needs to know which space the tensor lives in. A plausible coefficient can encode the wrong geometry.

# What canonicalisation establishes

- It identifies equivalent forms under supported index permutations and dummy-index relabelling.
- Metric contraction applies the declared metric relations.
- A zero residual supports the identity within those assumptions and transformations.
- A nonzero residual may need further identities or simplification. It does not always disprove equivalence.

Bianchi identities and dimension-dependent identities may require additional operations. Check the algorithm's scope.

Source: xTensor documentation Cadabra canonicalise: an explicit example of algorithm scope

# Verification uses different kinds of evidence

1. **Structure**: matching free indices, correct index variance, units and symmetries.
2. **Analytic limit**: flat spacetime, a known background or a lower perturbative order.
3. **Independent calculation**: hand derivation or components in a simple geometry.
4. **Residual**: simplify the difference between candidate and reference expressions under the same conventions.

A numerical spot check can expose an error. It does not prove a general tensor identity. Repeating the same assumptions in another program can repeat the same mistake.

# Research example: perturbing the inverse metric

Let

$$g_{ab}(\epsilon) = \bar{g}_{ab} + \epsilon h_{ab}, \qquad h^{ab} = \bar{g}^{ac}\bar{g}^{bd}h_{cd}.$$

Then

$$g^{ab}(\epsilon) = \bar{g}^{ab} - \epsilon h^{ab} + \epsilon^2 h^a{}_c h^{cb} + \mathcal{O}(\epsilon^3).$$

- Check the product $g^{ac}(\epsilon)g_{cb}(\epsilon) = \delta^a{}_b$ through the retained order.
- A wrong sign at first order fails immediately.
- xPert automates perturbative bookkeeping. xPand adds cosmological background structure and gauge choices.

Source: Pitrou, Roy and Umeh (2013), xPand. See also xact.es/xPert/

# An LLM-assisted calculation you can audit

**Example request**

> *Using xTensor, define a 4D Lorentzian metric, symmetric $S_{ab}$ and antisymmetric $F_{ab}$. Give a minimal script for $S_{ab}F^{ab}$. Explain the declarations and add a test with a generic rank-two tensor. Identify any command you are unsure about.*

1. Inspect definitions and check commands against documentation.
2. Execute in a clean session and read warnings and messages.
3. Compare with the hand argument and a test that should fail.
4. Save the code and actual outputs with the conventions.

If the LLM runs tools itself, inspect its executed code and tool output too.

# A reusable research calculation

- A clean script or notebook runs from the beginning.
- It records dimensions, sign conventions, assumptions and approximation order.
- It records software and package versions, plus input data when needed.
- It includes checks with known expected results and exports the final expression.
- It avoids unnecessary expansion and simplifies in stages when expressions grow.

**A useful standard for assessment:** explain every declaration, repair one incorrect assumption, and demonstrate an independent check. LLM assistance can be allowed and documented.

# Discussion: what would convince you?

An LLM returns a long second-order cosmological expression and says it checked the result.

- Which conventions and gauge choices do you need?
- What executed evidence would you request?
- Which simple limit would you test first?

**Why learn one tool?** To turn a proposed answer into a calculation you understand, can reproduce and can challenge.

# Optional reference: setup and demonstration

1. Install Mathematica or Wolfram Engine with suitable access.
2. Follow the official xAct installation instructions and test package loading before class.
3. Start a fresh kernel and evaluate `Computation_tensor_algebra_2026_demo.wl`.
4. Read each test result and any messages. Save the actual outputs and versions.

The slides include expected results, so the mathematical exercise also works without a live installation.

Source: xAct download and installation instructions

# Optional reference: threading and slicing

**Threading:** a timelike congruence $u^a$ describes observer worldlines.

$$h_{ab} = g_{ab} + u_a u_b, \qquad u^a u_a = -1.$$

Its local rest spaces form orthogonal hypersurfaces only when the congruence has zero vorticity (locally, by Frobenius).

**Slicing:** spacelike hypersurfaces have a unit timelike normal $n^a$. In adapted coordinates,

$$\mathrm{d}s^2 = -N^2 \mathrm{d}t^2 + h_{ij}(\mathrm{d}x^i + N^i \mathrm{d}t)(\mathrm{d}x^j + N^j \mathrm{d}t).$$

The lapse $N$ and shift $N^i$ describe the relation between the coordinates and the normal observers.

# Optional reference: ADM checks

With $K_{ij} = -\frac{1}{2}\mathcal{L}_n h_{ij}$, $c = 1$, and Einstein's equation $G_{ab} + \Lambda g_{ab} = 8\pi G T_{ab}$,

$$\partial_t h_{ij} = -2NK_{ij} + D_i N_j + D_j N_i,$$

$$^{(3)}R + K^2 - K_{ij}K^{ij} = 16\pi G\rho + 2\Lambda,$$

$$D_j K^j{}_i - D_i K = 8\pi G j_i.$$

Here $\rho = T_{ab} n^a n^b$ and $j_i = -h_i{}^a T_{ab} n^b$. These equations provide useful checks of lapse factors, units and sign conventions in a component calculation.

Source: Gourgoulhon, 3+1 formalism and bases of numerical relativity

# Optional reference: Raychaudhuri check

For a unit timelike congruence, with $A_a = u^b \nabla_b u_a$, $\Theta = \nabla_a u^a$, and

$$\nabla_b u_a = -u_b A_a + \tfrac{1}{3}\Theta h_{ab} + \sigma_{ab} + \omega_{ab},$$

the expansion obeys

$$u^c \nabla_c \Theta = -\tfrac{1}{3}\Theta^2 - \sigma_{ab}\sigma^{ab} + \omega_{ab}\omega^{ab} - R_{ab}u^a u^b + \nabla_a A^a.$$

$$\nabla_a A^a = D_a A^a + A_a A^a.$$

The vorticity contraction has a positive sign in these conventions. Geodesic, irrotational limits give useful checks before a full symbolic derivation.

Source: Ellis and van Elst, Cosmological models (congruence kinematics)

# Documentation and further reading

- xAct package catalogue and documentation, especially xTensor, xCoba and xPert.
- xAct community examples: inspect assumptions before adapting a notebook.
- Pitrou, Roy and Umeh, xPand, CQG 30 (2013) 165002.
- Cadabra and canonicalisation documentation.
- SymPy abstract tensor documentation.
- Gou et al., ToRA, ICLR 2024: an example of integrating LLMs with mathematical tools.

Documentation links checked for this 2026 update. Benchmark results on other mathematical tasks do not establish reliability for research tensor calculations.